Sri Lanka Institute of Information Technology

BSc Honors in Information Technology

Specializing in Cyber Security

IE3062 - Data and Operating System Security

Group Assignment
IE3062_25_017

Student Register Number	Student Name
IT22 2300 10	UMAYANGA H L A
IT22 2498 52	KARUNARATHNA P.M.T.L
IT22 0836 78	SAHAN H.P.T
IT22 1215 92	HERATH H.M.C.H.K

Table of Contents

Scenario 3
TASK 1 3
Compare and contrast 5 Linux distributions 3
Linux Distribution Comparison 5
Recommendation 5
Ubuntu Installation 7
Common Security Risks 10
10 Configurations to Harden Ubuntu Server 10
TASK 2 17
Install the Oracle DBMS on the machine on the Linux distribution 17
Five configurations to harden Ubuntu. 18
1. Secure SSH Access 18
2. Setup Firewall (UFW) 19
3. Install and Configure Fail2ban 20
4. Automatic Security Updates 21
5. AppArmor for Mandatory Access Control 21
ER Diagram 22
Tables And Data 23
TASK 3 – Big Data Security 33
Security Requirements of Big Data vs. Traditional Databases 34
Common Attacks on Big Data Systems and Mitigations 35
Critical Discussion 36
References 37
Appendix 38
SQL/PLSQL scripts 38

Scenario

You have been hired as a junior system admin at a small company that is looking to set up a new server to run a database for their new HR system. The head of the IT team has requested you to install and properly configure both the OS and a suitable DBMS on the server to use for this need.

TASK 1

Compare and contrast 5 Linux distributions and select and justify the most suitable one to be used as the OS for this server.

1. Ubuntu Desktop

Advantages	Disadvantages
User-Friendly GUI: Comes with the GNOME desktop environment, which is easy to use.	**More Resource-Intensive**: Requires more system resources than a server-only installation.
Extensive Community Support: Large online community for troubleshooting and guidance.	**Suitable For:** Those who prefer an easy to-use GUI and don't mind using slightly more system resources.
LTS Versions: Long-term support versions ensure stability and regular security updates	
Compatibility: Compatible with various DBMS options and server software.	

2. Fedora Workstation

Advantages	Disadvantages
Easy Installation: This comes with the GNOME desktop as the default, which makes it suitable for individuals who favor a graphical user interface.	**Suitable For:** Individuals seeking the most recent software featuring an intuitive graphical user interface
Security: Enhanced security is provided by SELinux	**Frequent Updates:** Regular updates are necessary, but they may not be suitable for a stable server
Cutting-Edge Software: Offers the most recent functionalities and enhancements.	

3. CentOS Stream with GNOME Desktop

Advantages	Disadvantages
Enterprise-Grade with GUI: The GNOME desktop environment can be added to CentOS during installation to provide a graphical user interface option	**Learning Curve:** Slightly harder to handle for newcomers in comparison to Ubuntu, particularly due to its increased complexity.
Robustness: Stable, which makes it appropriate for servers, also provides the adaptability of a graphical interface.	**Suitable For:** Enterprise-ready environment with GUI support is required by users.
Security: Security tools are integrated, such as SELinux.	

4. OpenSUSE Leap with KDE Plasma or GNOME

Advantages	Disadvantages
Flexible GUI Choices: Enables the installation of KDE Plasma, GNOME, or other desktop environments.	**Suitable For:** Users requiring a reliable server environment with a graphical user interface for easier administration.
Stable and Secure: Provides a secure-focused platform with a stable foundation.	**Moderate Learning Curve:** The YAST tool makes it easier, although openSUSE is somewhat more complex than Ubuntu.
YAST Control Center: System management and configuration are made simpler.	

5. Linux Mint

Advantages	Disadvantages
User-Friendly: Based on Ubuntu, it features a conventional and user-friendly graphical user interface.	**Not Primarily Server-Oriented:** Designed mainly for use on desktops, although it can also be set up for server-related activities.
Cinnamon Desktop: Provides a lightweight and intuitive desktop environment.	**Suitable For:** Users seeking a very user-friendly graphical user interface that does not require a server operating system focused on enterprise needs.
Compatibility: Works well with different types of software, including a variety of choices for managing database systems.	

Linux Distribution Comparison

Distribution	Release Cycle	Package Manager	Commercial Support	Community support
Ubuntu Desktop	6 Months	APT	Yes	Large
Fedora Workstation	6 Months	DNF	Yes	Large
CentOS Stream	6-8 Months	YUM	Yes	Large
OpenSUSE Leap	1 Year	Zypper	Yes	Large
Linux Mint	6 Months	APT	Yes	Large

Recommendation

When setting up a new server for a database to support an HR system in a small company, it's best to go with Ubuntu Server with a GUI (using Ubuntu Desktop) for security and database management.

1. **Ease to Use with GUI:**

   - Ubuntu Desktop provides a user-friendly GNOME GUI, simplifying server and database management tasks, even for those unfamiliar with command-line operations.

   - Ubuntu Desktop offers a user-friendly GNOME GUI, simplifying server and database management tasks for users less familiar with command-line operations.

2. **Extensive Software and Community Support:**

   - Ubuntu provides a wide range of software packages, including robust and secure database management systems (DBMS) such as MySQL, PostgreSQL, and MariaDB.

   - With plenty of help from the community, you can easily find guides, forums, and documentation to help you secure and manage your database server effectively.

3. **Security Features:**

- **Regular Security Updates:** Ubuntu's Long-Term Support (LTS) versions offer frequent security updates and bug fixes, which are essential for maintaining a secure server environment.

- **Built-In Firewall (UFW):** Ubuntu is equipped with the Uncomplicated Firewall (UFW), which simplifies the process of establishing and adjusting firewall regulations using either the command line or graphical user interface (GUI) tools such as GUFW.

- **AppArmor:** a security module included in Ubuntu, imposes strict access controls on programs. This is crucial for limiting database access and reducing potential points of attack.

- **Encrypted Database Support:** Supports secure database solutions with encryption to provide an additional level of protection for confidential HR information.

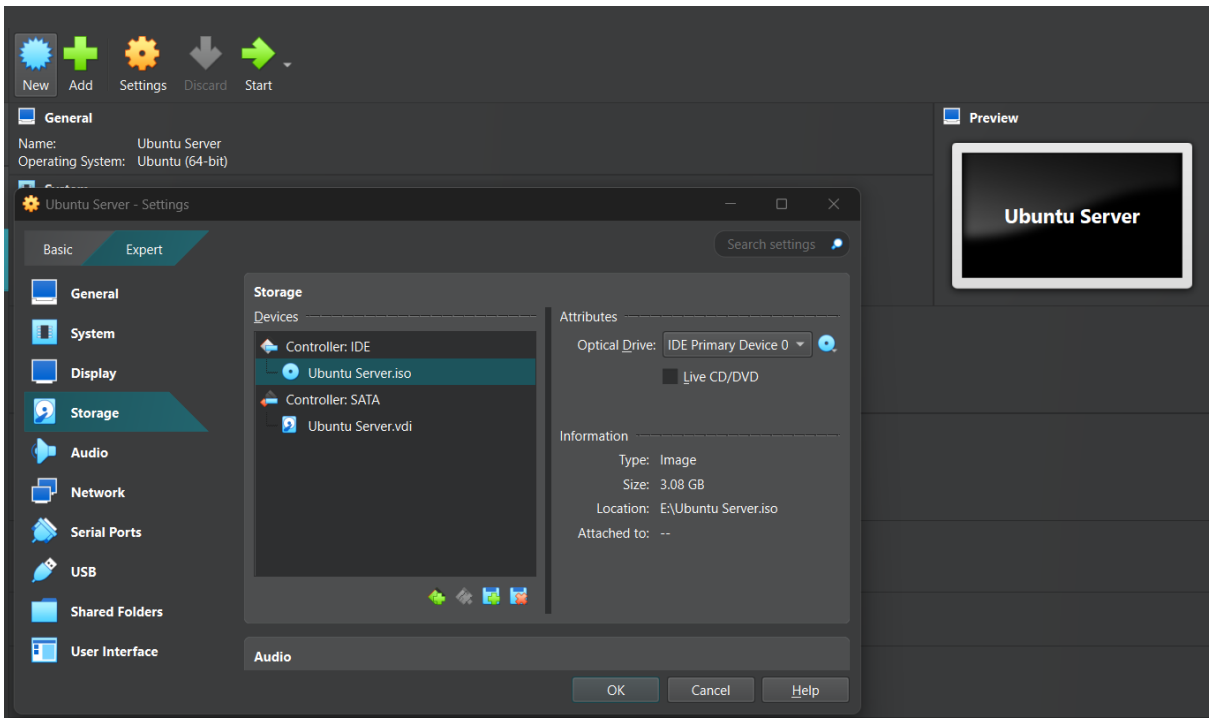
4. **Database Management Tools:**

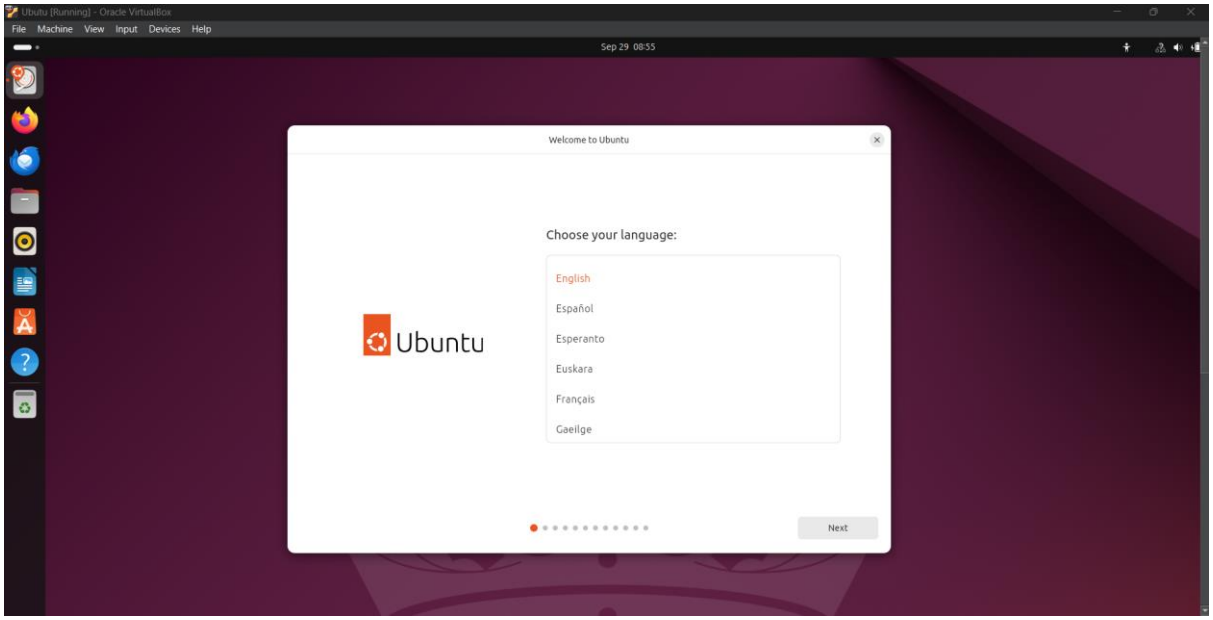
- Ubuntu offers graphical database management tools such as phpMyAdmin for MySQL and pgAdmin for PostgreSQL, which make it easier to manage and secure databases through the graphical user interface.

- Automated security updates for the database management system (DBMS) can be easily configured, reducing the risk of vulnerabilities.

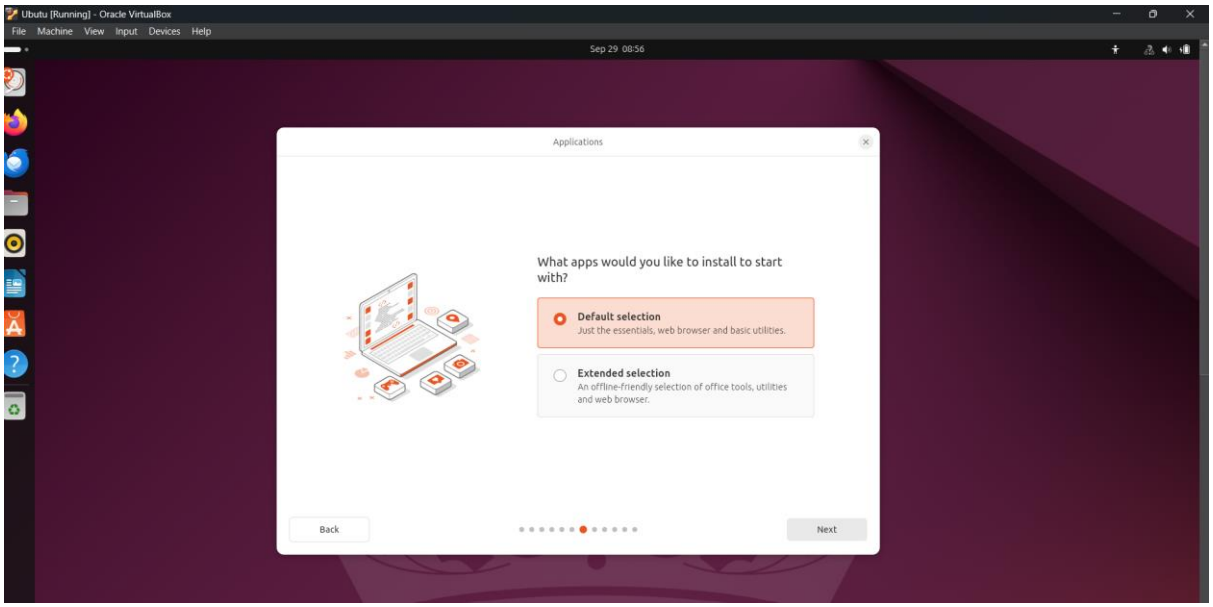
Ubuntu Installation

Please ensure to start the new VM (Machine > Start) after setting it up for installing Ubuntu on VirtualBox. You can use the ISO installation image of Ubuntu to start the VM. Once your computer boots up, you'll see the Welcome screen. To display information in the installation interface, select a language from the left pane. In this case, we'll choose English. Then, select Ubuntu Install.

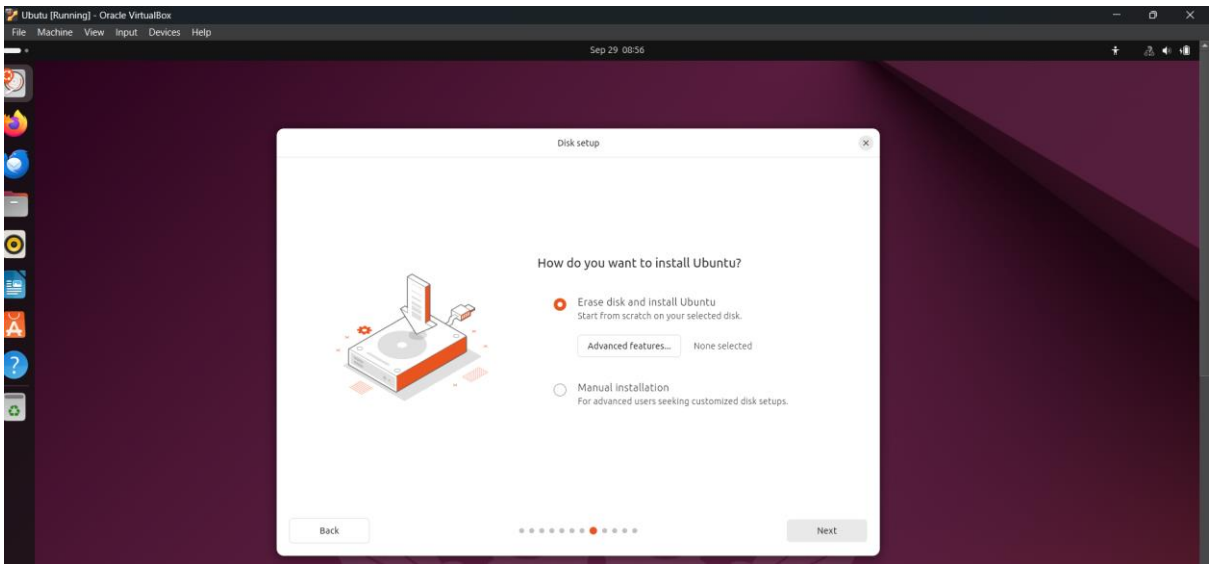




Now, let's begin setting up your keyboard layout. First, choose an English keyboard layout, and then click on the Continue button. Following that, we will select the regular installation option, and we'll also have the option to download updates while installing Ubuntu.

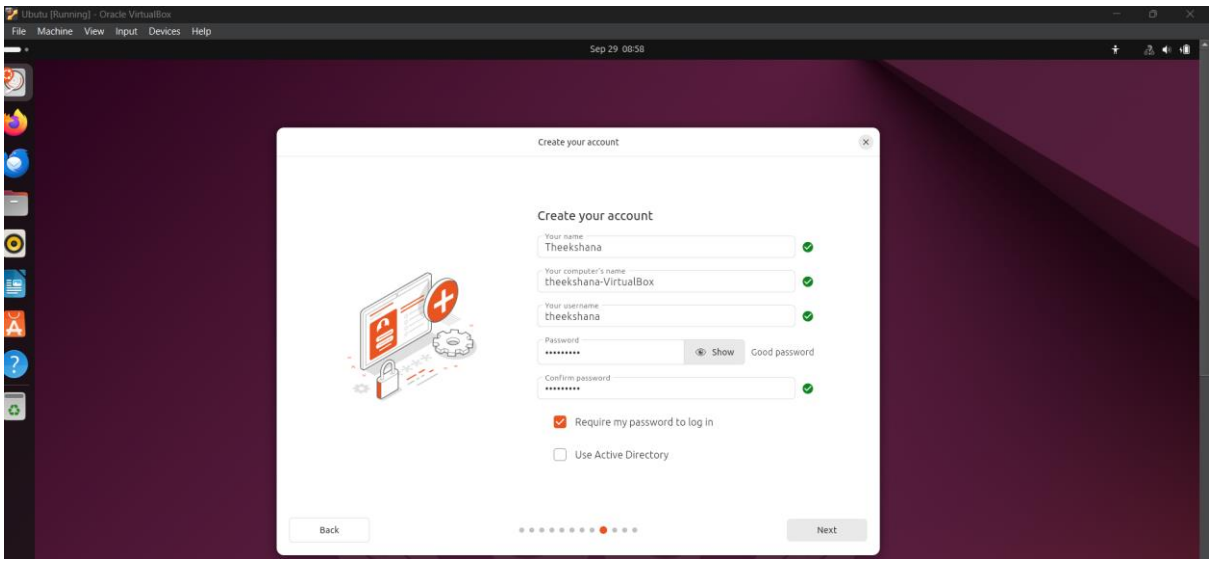


Press the Next button and choose Erase disk and install Ubuntu. When you're ready, click on Install Now to proceed.

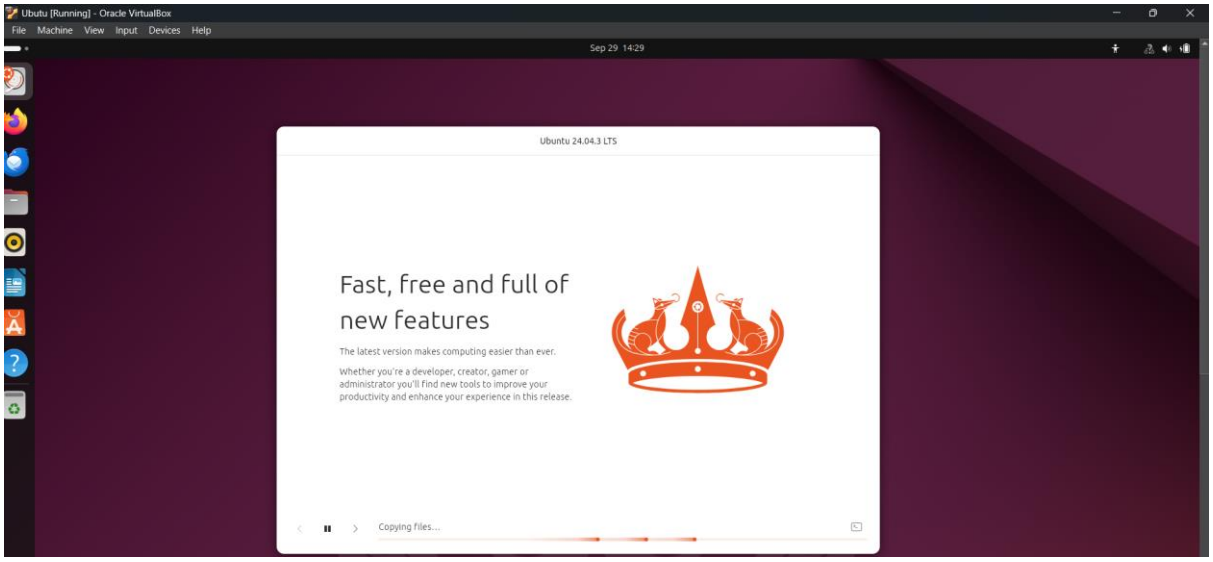


Click continue then proceed to select the location as Colombo to set the time zone and regional settings. Then click the continue button “again”

In the "Who are you?" section, input your username, computer's name, and create the password. After that, click the continue button



The completion of the installation process will require you to wait.



Once the installation is finished, a notification window will appear. It is necessary to restart your VirtualBox VM running Ubuntu.

Common Security Risks

1. Unsecured SSH Access: Attackers may take advantage of vulnerable SSH configurations or attempt to guess default credentials.

2. Unnecessary Services Running: Having open ports and services expands the potential targets for cyber-attacks.

3. Default User Privileges: Watch out - if user privileges are set up incorrectly, it could lead to a serious issue like privilege escalation.

4. Weak Firewall Rules: The server is at risk of external attacks due to the absence of firewall configuration.

5. Outdated Software and Kernel: Outdated packages and the kernel could harbor unpatched vulnerabilities. Keep your system up to date.

6. Unsecured Network Services: Exposing sensitive services such as databases to public networks can make them vulnerable to attacks.

7. Weak Password Policies: Sometimes people use easily guessable passwords, which can result in unauthorized access.

8. No File Integrity Monitoring: It's challenging to spot tampered files without proper integrity monitoring in place.

9. Unrestricted Sudo Access: Using sudo too often without proper monitoring can pose a serious security risk.

10. Lack of Data Encryption: Data is vulnerable to exposure during transmission and storage without encryption.

10 Configurations to Harden Ubuntu Server

01. Encrypt Data in Transit and at Rest:

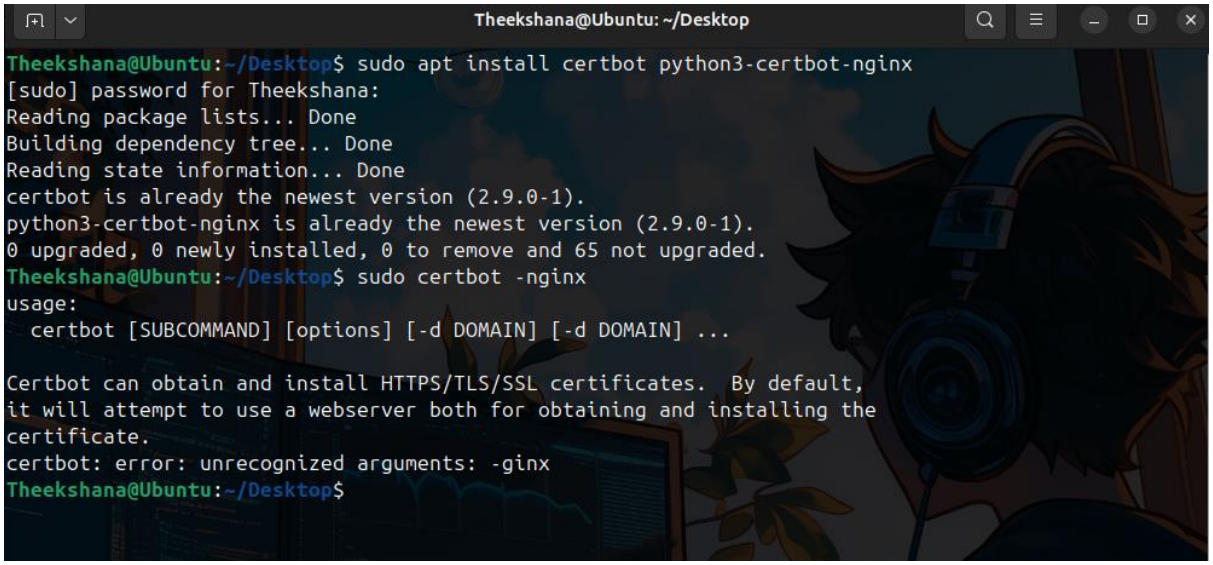
Action:

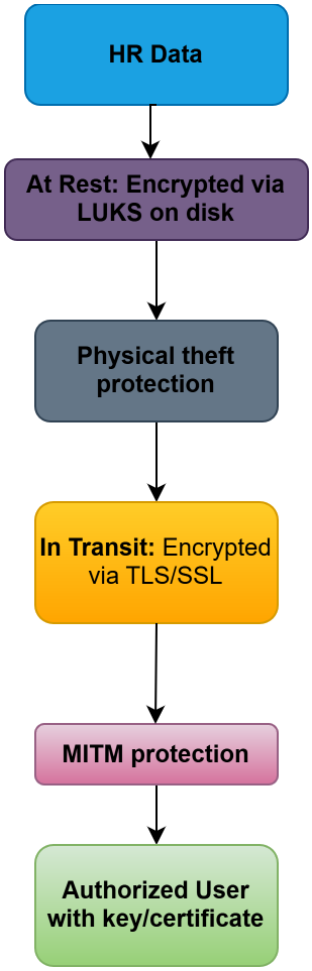
- Enable SSL/TLS for web services and databases.
- Use LUKS to encrypt disks containing sensitive data.

Why: Make sure that confidential information is encrypted while being transmitted and stored to safeguard it from interception or theft.

Protection Against: Data breaches, man-in-the-middle (MITM) attacks, and physical theft.

sudo apt install certbot python3-certbot-nginx
sudo certbot –nginx





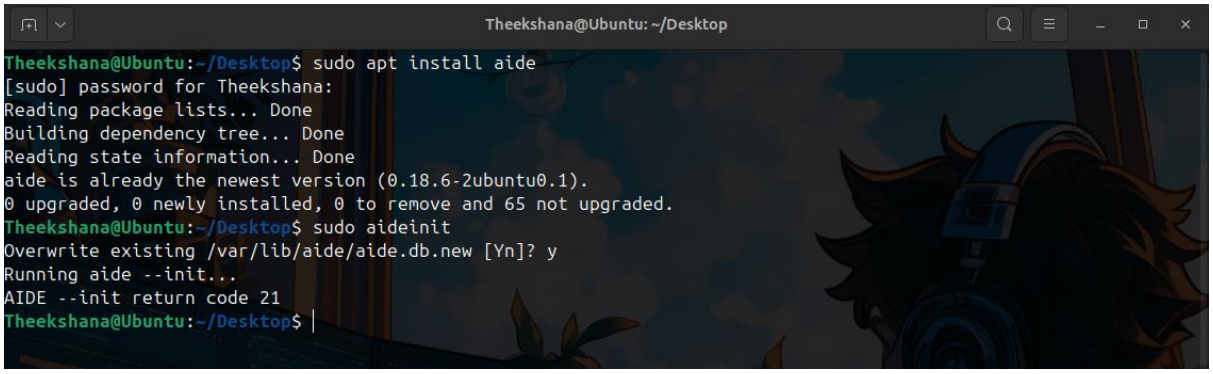
2. Configure File Integrity Monitoring (FIM):

Utilize AIDE by running "sudo apt install aide" in order to supervise critical system files and detect any unauthorized alterations.

Why: Monitors modifications to crucial files that could signal a security breach.

Protection Against: Rootkits, malware, and unauthorized changes.

sudo apt install aide
sudo aideinit
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db



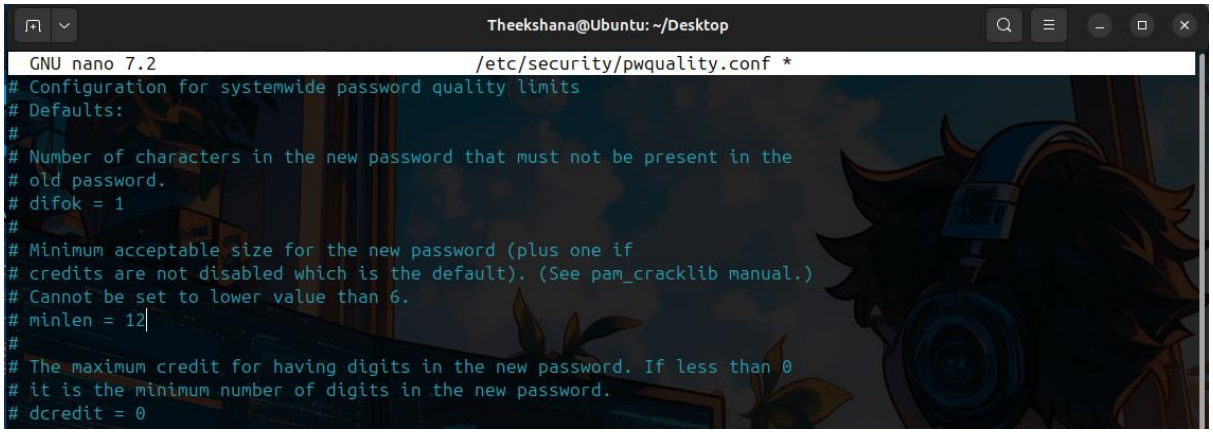
3.Enforce Strong Password Policies:

Adjust /etc/security/pwquality.conf to ensure that there is a minimum required password length and complexity.

Why: Helps users generate complex passwords that are difficult to predict.

Protection Against: Weak password attacks and credential stuffing.

Sudo nano /etc/security/pwquality.conf
 # Enforce strong password policies like minlen=12



4.Set Up Fail2Ban:

Set up Fail2Ban (use the command sudo apt install fail2ban) and set it up to monitor critical services including SSH.

Why: Automatically prevents access from specific IPs following a specific number of unsuccessful login attempts.

Protection Against: Brute-force and dictionary attacks.

sudo apt install fail2ban
sudo systemctl enable fail2ban
sudo systemctl start fail2ban

5.Restrict Sudo Access: Please modify the /etc/sudoers file to limit sudo access to only essential users and use the NOPASSWD option judiciously.

Why: Prevents unauthorized users from gaining elevated privileges.

Protection Against: Privilege escalation attacks.

sudo visudo
 # Modify the sudoers file to restrict access

```
Theekshana@Ubuntu: ~/Desktop
GNU nano 7.2                                  /etc/sudoers.tmp
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
#
# See the man page for details on how to write a sudoers file.
#
Defaults        env_reset
Defaults        mail_badpass
Defaults        secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"

# This fixes CVE-2005-4890 and possibly breaks some versions of kdesu
# (#1011624, https://bugs.kde.org/show_bug.cgi?id=452532)
Defaults        use_pty
                                        [ Read 57 lines ]
^G Help    ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo
^X Exit    ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

6.Disable Unnecessary Services:

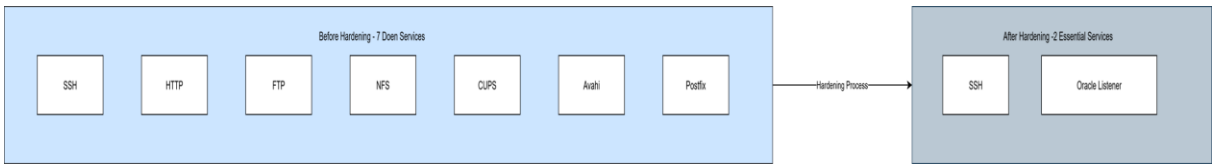
Use systemctl list-units --type=service to display a list of services and use sudo systemctl disable to deactivate any services that are not needed.

Why: Minimizes the attack surface by running only necessary services.

Protection Against: Exploits targeting unnecessary or vulnerable services.

sudo systemctl list-units --type=service
sudo systemctl disable

```
Theekshana@Ubuntu: ~/Desktop
Theekshana@Ubuntu:~/Desktop$ sudo systemctl list-units --type=service
  UNIT                                  LOAD   ACTIVE SUB     DESCRIPTION
  accounts-daemon.service               loaded active running Accounts Service
  alsa-restore.service                  loaded active exited  Save/Restore Sound Card State
  apparmor.service                      loaded active exited  Load AppArmor profiles
  apport.service                        loaded active exited  automatic crash report generation
  avahi-daemon.service                  loaded active running Avahi mDNS/DNS-SD Stack
  cloud-config.service                  loaded active exited  Cloud-init: Config Stage
  cloud-final.service                   loaded active exited  Cloud-init: Final Stage
  cloud-init-local.service              loaded active exited  Cloud-init: Local Stage (pre-network)
  cloud-init.service                    loaded active exited  Initial cloud-init job (metadata service crawler)
  colord.service                        loaded active running Manage, Install and Generate Color Profiles
  console-setup.service                 loaded active exited  Set console font and keymap
  cron.service                          loaded active running Regular background program processing daemon
  cups-browsed.service                  loaded active running Make remote CUPS printers available locally
  cups.service                          loaded active running CUPS Scheduler
  dbus.service                          loaded active running D-Bus System Message Bus
  fail2ban.service                      loaded active running Fail2Ban Service
  fwupd.service                         loaded active running Firmware update daemon
```



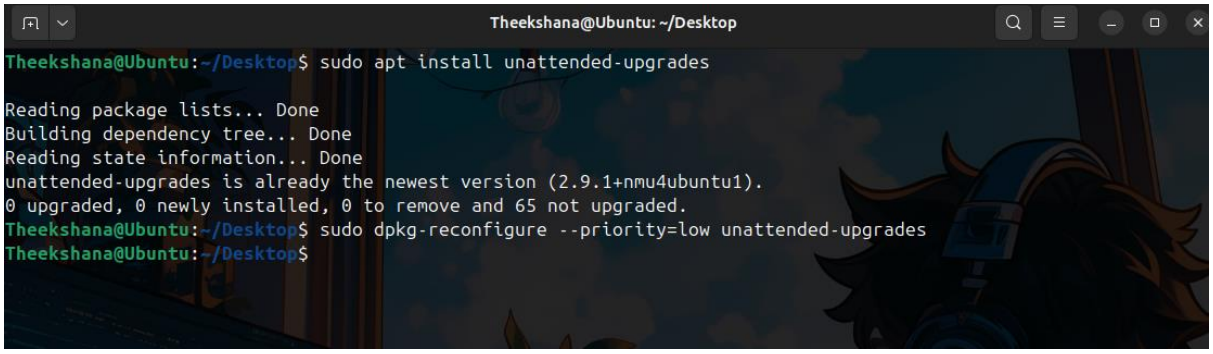
7.Keep the System and Software Updated:

Configure unattended-upgrades to enable automatic security updates (sudo apt install unattended-upgrades).

Why: Patching recognized vulnerabilities promptly decreases the likelihood of exploitation.

Protection Against: Exploits targeting unpatched vulnerabilities.

sudo apt install unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades



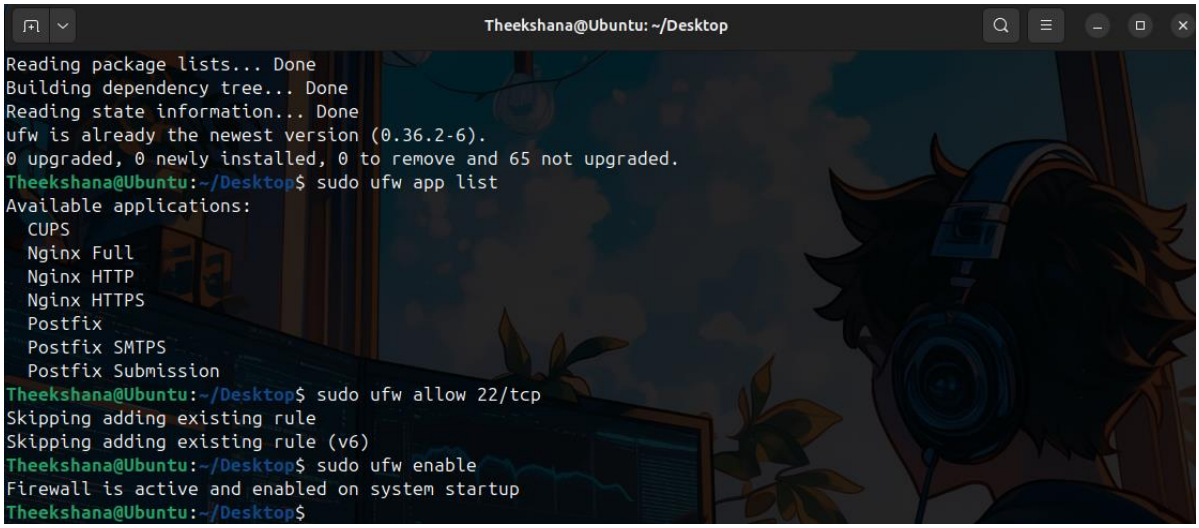
8.Install and Configure a Firewall (UFW):

Install UFW using the command "sudo apt install ufw" and set up rules to allow only necessary services, such as "sudo ufw allow OpenSSH" and deny all other incoming connections.

Why: Limits access to only essential services to reduce the attack surface.

Protection Against: Unauthorized access attempts and port scanning.

sudo apt install ufw
sudo ufw allow OpenSSH
sudo ufw enable



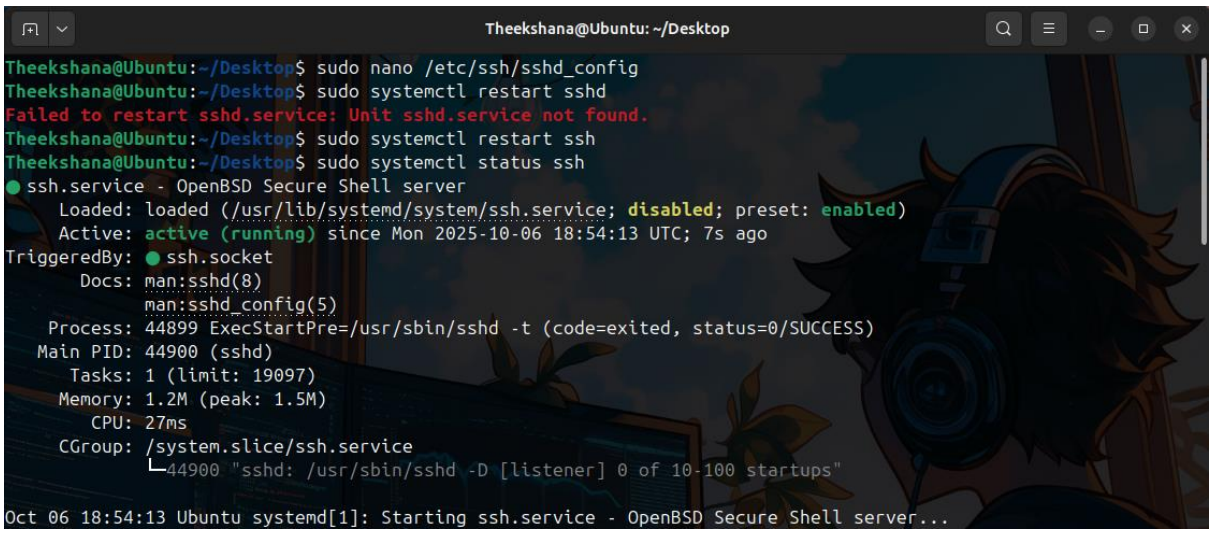
9.Enforce Strong SSH Authentication:

Remember to use SSH key-based authentication instead of passwords by updating the PasswordAuthentication setting to "no" in the /etc/ssh/sshd_config file.

Why: SSH keys are much harder to crack than passwords.

Protection Against: Brute-force attacks on SSH accounts.

sudo nano /etc/ssh/sshd_config
 # Set PasswordAuthentication to no
PasswordAuthentication no
sudo systemctl restart sshd



10.Disable Root Login via SSH:

Make changes to /etc/ssh/sshd_config by setting PermitRootLogin to no.

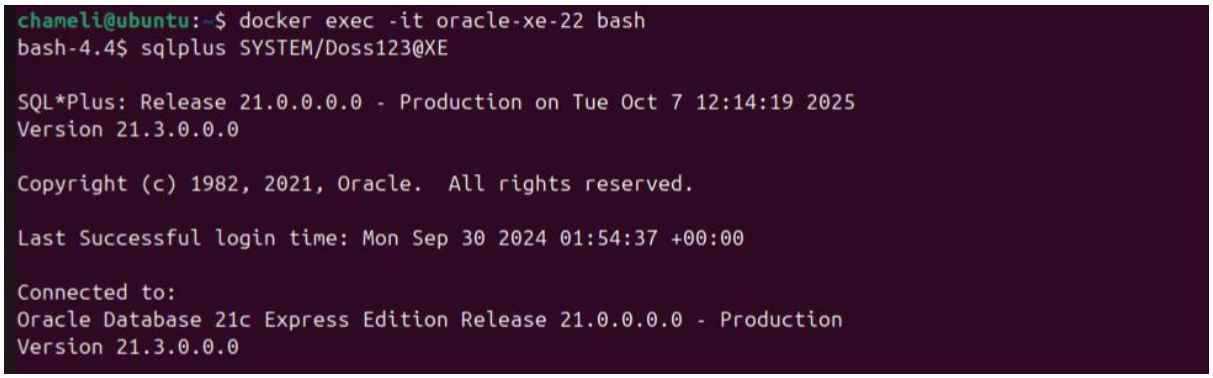
Why: Stop attackers from trying to brute-force the root account over SSH.

Protection Against: SSH brute-force attacks targeting the root user.

sudo nano /etc/ssh/sshd_config
 # Set PermitRootLogin to no
PermitRootLogin no
sudo systemctl restart sshd

TASK 2

Install the Oracle DBMS on the machine on the Linux distribution



Five configurations to harden Ubuntu.

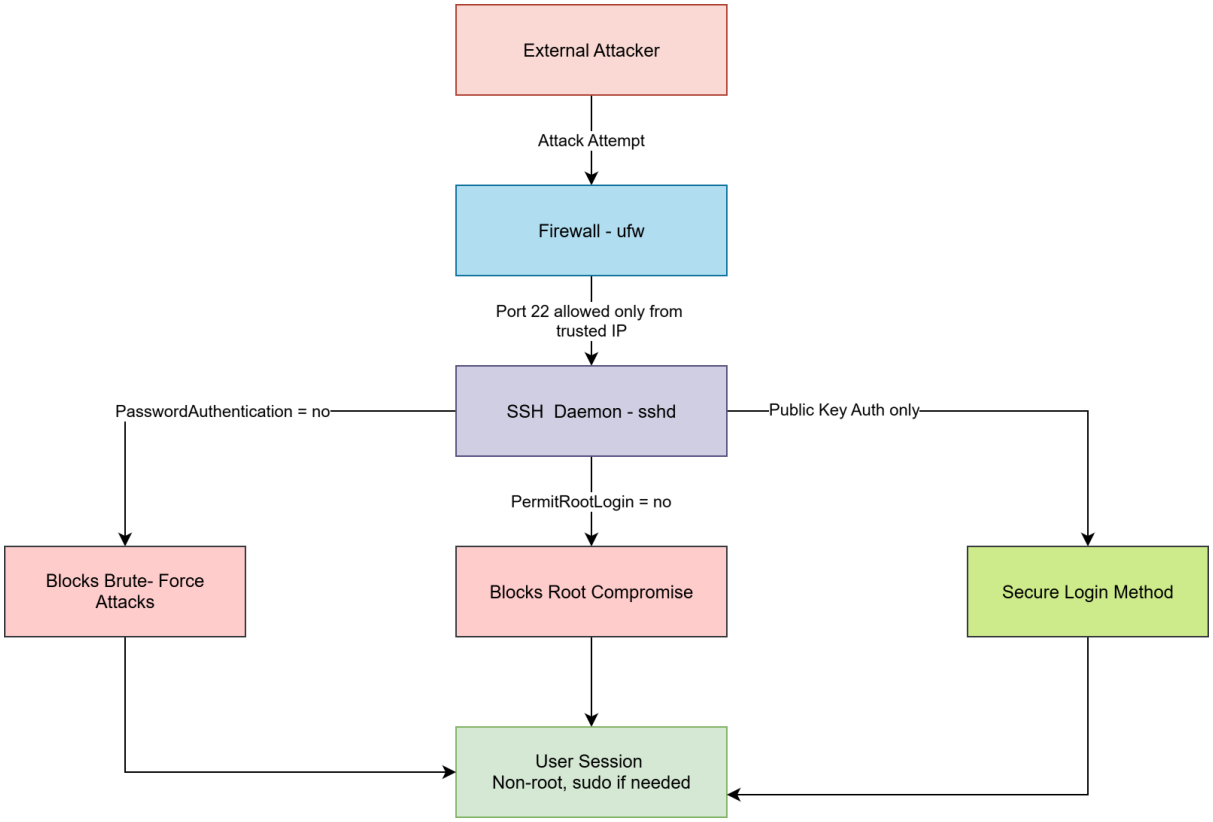
1. Secure SSH Access

Configuration:

- Disable root login.
- Disable password authentication and use SSH keys instead.
- Change the default SSH port.

Why does it add security:

SSH is one of the most common entry points for attackers. By disabling root login, even if an attacker compromises SSH, they cannot directly gain full control of the system. SSH keys are much harder to crack compared to passwords, providing stronger authentication. Changing the default SSH port reduces exposure to automated scans and brute-force attempts that typically target port 22.



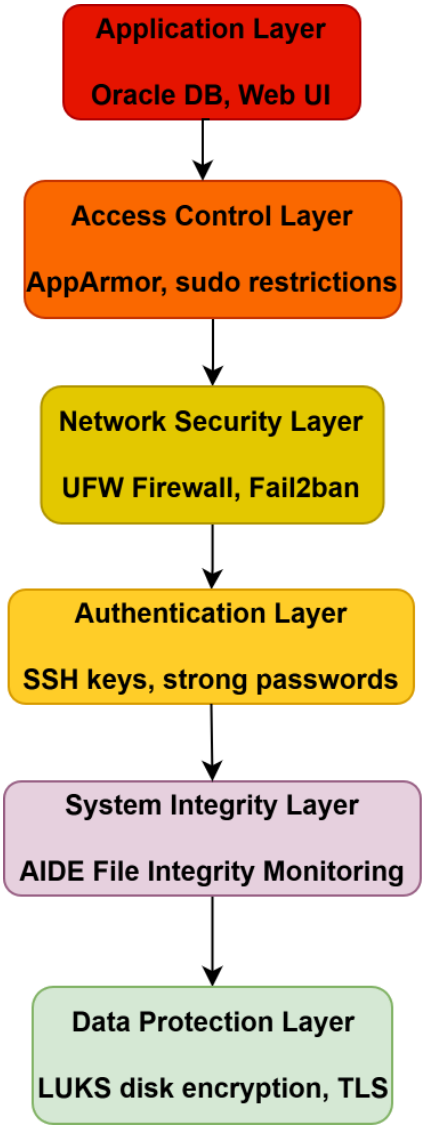
2. Setup Firewall (UFW)

Configuration:

- Enable and start the UFW firewall.
- Set the default rule to deny all incoming connections.
- Allow only necessary traffic such as SSH (from trusted IPs) and Oracle's database port if required.

Why does it add security:

A firewall acts as the first line of defense by controlling what traffic can enter or leave the system. Denying all incoming traffic by default and only allowing essential services minimizes the attack surface and prevents unauthorized access through unused or unnecessary ports.



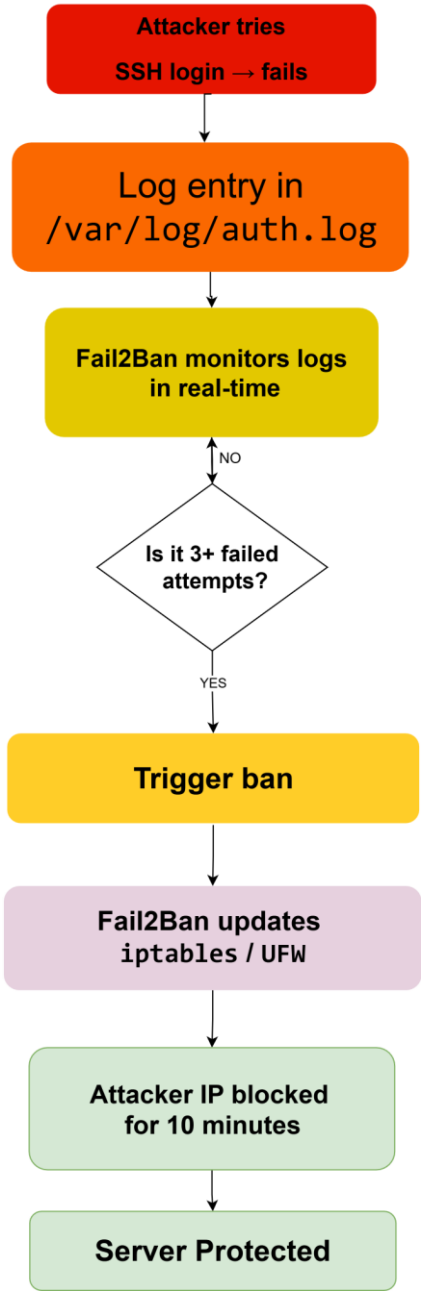
3. Install and Configure Fail2ban

Configuration:

- Install Fail2ban.
- Configure it to monitor SSH logs (and Oracle listener logs if used).
- Block IP addresses after repeated failed login attempts.

Why does it add security:

Fail2ban protects the server against brute-force login attacks by detecting repeated failed login attempts in logs. When suspicious behavior is detected, it automatically bans the attacker's IP address temporarily or permanently, reducing the risk of compromise through brute-force guessing.



4. Automatic Security Updates

Configuration:

- Enable unattended-upgrades on Ubuntu.
- Ensure that security patches are applied automatically as soon as they are available.

Why does it add security:

Outdated software is one of the most common ways attackers exploit systems. By enabling automatic updates, Ubuntu will always have the latest security patches installed without waiting for manual intervention, closing vulnerabilities quickly before they can be exploited.

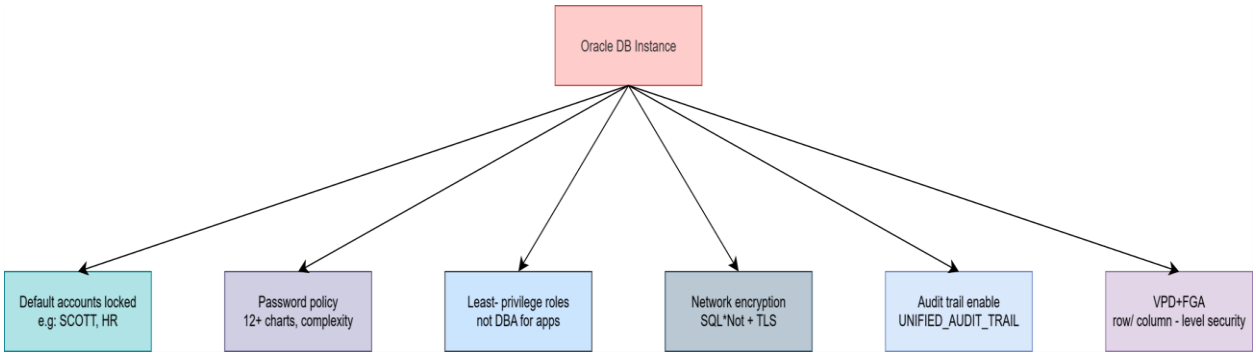
5. AppArmor for Mandatory Access Control

Configuration:

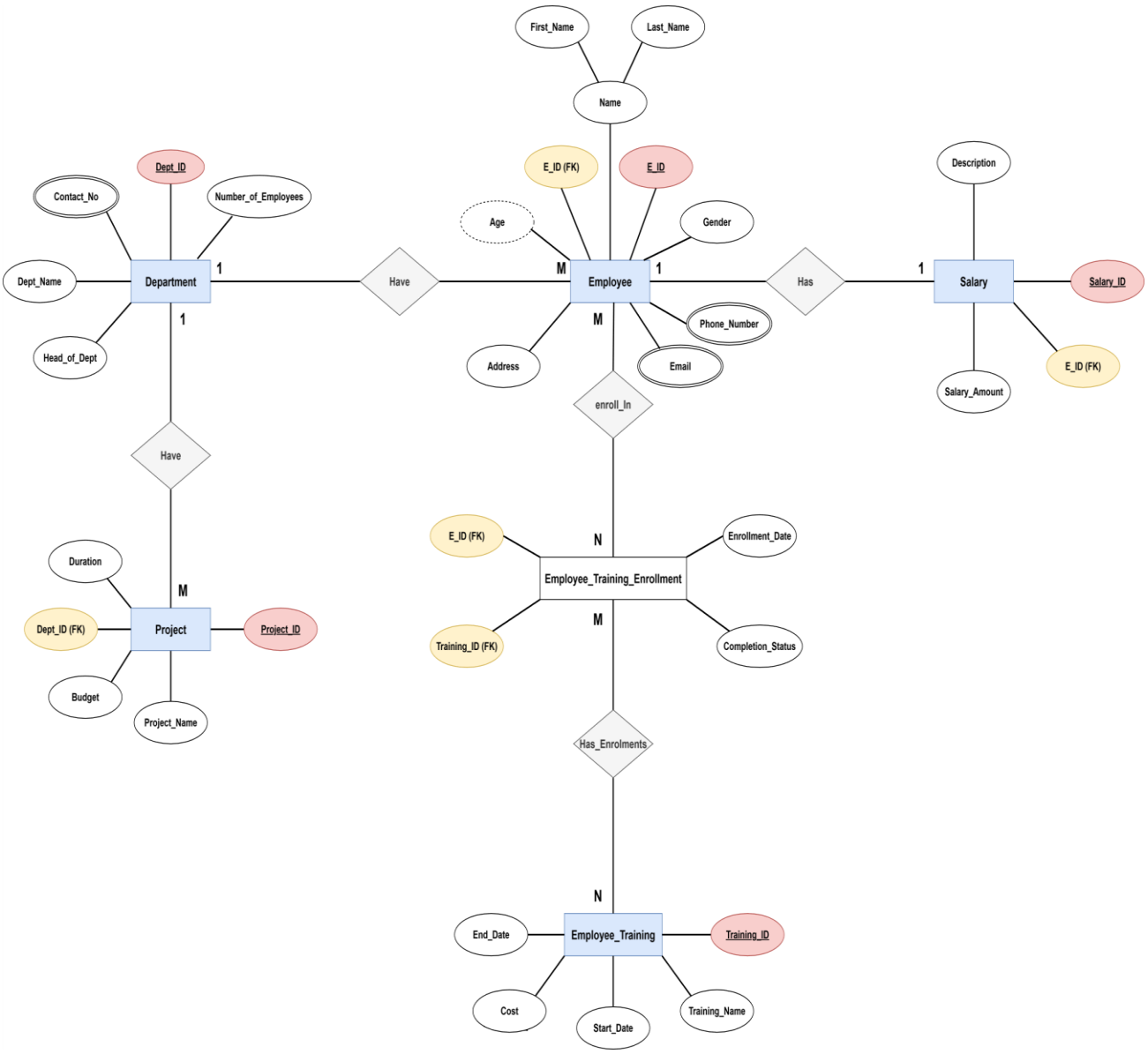
- Enable and configure AppArmor profiles for critical services (e.g., SSH, Oracle database).
- Restrict applications to only the files and resources they absolutely need.

Why does it add security:

Even if an attacker compromises an application, AppArmor limits what the application can do. It prevents unauthorized file access, privilege escalation, and lateral movement within the system. This provides an additional layer of security beyond traditional user permissions.



ER Diagram


First_Name
Last_Name
Name
Dept_ID
Contact_No
Number_of_Employees
E_ID (FK)
E_ID
Description
Age
Gender
Dept_Name
Department
1
Have
M
Employee
1
Has
1
Salary
Salary_ID
1
M
Phone_Number
Head_of_Dept
Address
Email
E_ID (FK)
Salary_Amount
enroll_In
Have
E_ID (FK)
N
Enrollment_Date
Duration
Employee_Training_Enrollment
M
M
Dept_ID (FK)
Project
Project_ID
Training_ID (FK)
Completion_Status
Budget
Project_Name
Has_Enrolments
N
End_Date
Employee_Training
Training_ID
Cost
Start_Date
Training_Name

Tables And Data

```
SQL> CREATE TABLE Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50),
    Head_of_Dept VARCHAR(50),
    Contact_No VARCHAR(15),
    Number_of_Employees INT
);
  2    3    4    5    6    7
Table created.

SQL> CREATE TABLE Employee (
    E_ID INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Phone_Number VARCHAR(15),
    Email VARCHAR(100),
    Address VARCHAR(150),
    Age INT,
    Gender VARCHAR(10),
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);  2    3    4    5    6    7    8    9   10   11   12

Table created.
```

```
SQL> CREATE TABLE Salary (
    Salary_ID INT PRIMARY KEY,
    E_ID INT,
    Salary_Amount DECIMAL(10, 2),
    Description VARCHAR(100),
    FOREIGN KEY (E_ID) REFERENCES Employee(E_ID)
);  2    3    4    5    6    7

Table created.

SQL> CREATE TABLE Project (
    Project_ID INT PRIMARY KEY,
    Dept_ID INT,
    Project_Name VARCHAR(50),
    Duration INT,
    Budget DECIMAL (10, 2),
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);  2    3    4    5    6    7    8

Table created.

SQL> CREATE TABLE Employee_Training (
    Training_ID INT PRIMARY KEY,
    Training_Name VARCHAR(100),
    Cost DECIMAL(10, 2),
    Start_Date DATE,
    End_Date DATE
);  2    3    4    5    6    7

Table created.
```

```
SQL> CREATE TABLE Employee_Training_Enrollment (
    E_ID INT,
    Training_ID INT,
    Enrollment_Date DATE DEFAULT SYSDATE,
    Completion_Status VARCHAR(20) DEFAULT 'Enrolled',
    PRIMARY KEY (E_ID, Training_ID),
    FOREIGN KEY (E_ID) REFERENCES Employee(E_ID),
    FOREIGN KEY (Training_ID) REFERENCES Employee_Training(Training_ID)
);  2    3    4    5    6    7    8    9

Table created.
```

```
SQL> INSERT INTO Department VALUES (1, 'Human Resources', 'Anuradha Perera', '0112456789', 12);
INSERT INTO Department VALUES (2, 'Finance', 'Sanduni Jayasinghe', '0112789456', 20);
INSERT INTO Department VALUES (3, 'IT Support', 'Kasun Fernando', '0112987452', 15);
INSERT INTO Department VALUES (4, 'Software Development', 'Dilshan Weerakoon', '0112236789', 25);
INSERT INTO Department VALUES (5, 'Sales', 'Chathurika Abeykoon', '0112765432', 30);
INSERT INTO Department VALUES (6, 'Marketing', 'Pradeep Aluthgama', '0112987645', 18);
INSERT INTO Department VALUES (7, 'Procurement', 'Ruwan Kumara', '0112547896', 10);
INSERT INTO Department VALUES (8, 'Research', 'Sajani Ratnayake', '0112678912', 8);
INSERT INTO Department VALUES (9, 'Training', 'Malith Silva', '0112435678', 6);
INSERT INTO Department VALUES (10, 'Customer Service', 'Tharushi Senanayake', '0112893456', 22);
1 row created.
```

```
SQL> INSERT INTO Employee VALUES (1, 'Nimal', 'Perera', '0771234567', 'nimalp@sliit.lk', 'Colombo', 29, 'Male', 1);
INSERT INTO Employee VALUES (2, 'Kavindu', 'Fernando', '0712345678', 'kavinduf@gmail.com', 'Kandy', 26, 'Male', 2);
INSERT INTO Employee VALUES (3, 'Dilani', 'Jayasinghe', '0753456789', 'dilanij@yahoo.com', 'Galle', 32, 'Female', 3);
INSERT INTO Employee VALUES (4, 'Sajith', 'De Silva', '0764567890', 'sajithd@outlook.com', 'Negombo', 35, 'Male', 4);
INSERT INTO Employee VALUES (5, 'Isuri', 'Gunawardena', '0705678901', 'isurig@gmail.com', 'Matara', 28, 'Female', 5);
INSERT INTO Employee VALUES (6, 'Harsha', 'Kumara', '0776789012', 'harsha.k@abc.lk', 'Kurunegala', 40, 'Male', 6);
INSERT INTO Employee VALUES (7, 'Pavithra', 'Rathnayake', '0747890123', 'pavi.r@ymail.com', 'Anuradhapura', 27, 'Female', 7);
INSERT INTO Employee VALUES (8, 'Chamara', 'Madushanka', '0728901234', 'chamaram@gmail.com', 'Batticaloa', 31, 'Male', 8);
INSERT INTO Employee VALUES (9, 'Nadeesha', 'Wijesinghe', '0789012345', 'nadeesha.w@abc.com', 'Jaffna', 33, 'Female', 9);
```

```
SQL> INSERT INTO Project VALUES (1, 4, 'Payroll System', 6, 500000.00);
INSERT INTO Project VALUES (2, 4, 'Inventory Management', 8, 750000.00);
INSERT INTO Project VALUES (3, 2, 'Budget Planning', 4, 300000.00);
INSERT INTO Project VALUES (4, 6, 'Digital Marketing Campaign', 3, 200000.00);
INSERT INTO Project VALUES (5, 5, 'Sales Tracking System', 5, 400000.00);
INSERT INTO Project VALUES (6, 3, 'Helpdesk Automation', 6, 250000.00);
INSERT INTO Project VALUES (7, 7, 'Supplier Portal', 7, 350000.00);
INSERT INTO Project VALUES (8, 8, 'AI Research', 12, 1000000.00);
INSERT INTO Project VALUES (9, 9, 'Leadership Program', 2, 150000.00);
INSERT INTO Project VALUES (10, 10, 'Customer Feedback App', 4, 450000.00);
```

```
SQL> INSERT INTO Employee_Training (Training_ID, Training_Name, Cost, Start_Date, End_Date)
  2  VALUES (1, 'Leadership Training', 50000.00, DATE '2025-01-15', DATE '2025-01-20');
INSERT INTO Employee_Training VALUES (2, 'Advanced Excel', 20000.00, DATE '2025-02-10', DATE '2025-02-12');
INSERT INTO Employee_Training VALUES (3, 'Cybersecurity Basics', 60000.00, DATE '2025-03-05', DATE '2025-03-10');
INSERT INTO Employee_Training VALUES (4, 'Public Speaking', 25000.00, DATE '2025-04-01', DATE '2025-04-03');
INSERT INTO Employee_Training VALUES (5, 'Digital Marketing', 45000.00, DATE '2025-05-15', DATE '2025-05-20');
INSERT INTO Employee_Training VALUES (6, 'Java Programming', 70000.00, DATE '2025-06-01', DATE '2025-06-15');
INSERT INTO Employee_Training VALUES (7, 'Oracle Database', 90000.00, DATE '2025-07-01', DATE '2025-07-10');
INSERT INTO Employee_Training VALUES (8, 'Negotiation Skills', 30000.00, DATE '2025-08-12', DATE '2025-08-14');
INSERT INTO Employee_Training VALUES (9, 'Project Management', 80000.00, DATE '2025-09-05', DATE '2025-09-12');
INSERT INTO Employee_Training VALUES (10, 'Cloud Computing', 100000.00, DATE '2025-10-01', DATE '2025-10-07');
```

1. Database User Roles And Access Control.

System Administrator: Full access, including all system-level features. **Manager:** Can read and write data in the database, but has no system-level rights. **Executive:** Read-only access; can view data but cannot modify, insert, or delete.

```
SQL> CREATE USER system_admin IDENTIFIED BY AdminPass123;
GRANT DBA TO system_admin;
GRANT CONNECT, RESOURCE TO system_admin;

User created.

SQL>
Grant succeeded.

SQL>
Grant succeeded.
```

```
SQL> CREATE USER manager_user IDENTIFIED BY ManagerPass123;
GRANT CONNECT, RESOURCE TO manager_user;

GRANT SELECT, INSERT, UPDATE, DELETE ON Department TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Employee TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Project TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Salary TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Employee_Training TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Employee_Training_Enrollment TO manager_user;

User created.

SQL>
Grant succeeded.

SQL> SQL>
Grant succeeded.

SQL>
Grant succeeded.

SQL>
Grant succeeded.

SQL>
Grant succeeded.

SQL>
Grant succeeded.
```

```
SQL> CREATE USER executive_user IDENTIFIED BY ExecPass123;
GRANT CONNECT TO executive_user;

GRANT SELECT ON Department TO executive_user;
GRANT SELECT ON Employee TO executive_user;
GRANT SELECT ON Project TO executive_user;
GRANT SELECT ON Salary TO executive_user;
GRANT SELECT ON Employee_Training TO executive_user;
GRANT SELECT ON Employee_Training_Enrollment TO executive_user;

User created.

SQL>
Grant succeeded.

SQL> SQL>
Grant succeeded.

SQL>
Grant succeeded.

SQL>
Grant succeeded.
```

2. Create a view that allows Managers to view the personal and salary details.

Provides Managers with a consolidated view of the personal and salary details of the employees they oversee, ensuring controlled access to sensitive information.

```
SQL> CREATE OR REPLACE VIEW Manager_View AS
SELECT
    E.E_ID,
    E.First_Name,
    E.Last_Name,
    E.Phone_Number,
    E.Email,
    E.Address,
    E.Age,
    E.Gender,
    D.Dept_Name,
    S.Salary_Amount
FROM Employee E
JOIN Department D ON E.Dept_ID = D.Dept_ID
JOIN Salary S ON E.E_ID = S.E_ID;
  2    3    4    5    6    7    8    9   10   11   12   13   14   15
View created.
```

3. Manager Data Access Control Using VPD.

Restricts Managers to see only the employee and salary data they are authorized to access.

```
SQL> CREATE OR REPLACE FUNCTION MANAGER_USER.MANAGERVPDFN (
    schema_name IN VARCHAR2,
    object_name IN VARCHAR2
) RETURN VARCHAR2
AS
BEGIN
    RETURN 'Dept_ID IN (SELECT dept_id FROM Manager_Department WHERE oracle_username = USER)';
END;
/
  2    3    4    5    6    7    8    9
Function created.
```

```
SQL> BEGIN
     DBMS_RLS.ADD_POLICY(
         object_schema    => 'SYSTEM',
         object_name      => 'EMPLOYEE',
         policy_name      => 'EMP_VPD',
         function_schema  => 'MANAGER_USER',
         policy_function  => 'MANAGERVPDFN',
         statement_types  => 'SELECT'
     );
END;
/
  2    3    4    5    6    7    8    9   10   11  BEGIN
*
```

```
SQL> BEGIN
     DBMS_RLS.ADD_POLICY(
         object_schema    => 'SYSTEM',
         object_name      => 'EMPLOYEE',
         policy_name      => 'EMP_VPD_UPDATE',
         function_schema  => 'MANAGER_USER',
         policy_function  => 'MANAGERVPDFN',  2    3
         statement_types  => 'UPDATE'
     );
END;
/  4    5    6    7    8    9   10   11

PL/SQL procedure successfully completed.

SQL> BEGIN
     DBMS_RLS.ADD_POLICY(
         object_schema    => 'SYSTEM',
         object_name      => 'EMPLOYEE',
         policy_name      => 'EMP_VPD_DELETE',
         function_schema  => 'MANAGER_USER',
         policy_function  => 'MANAGERVPDFN',
         statement_types  => 'DELETE'
     );
END;
/  2    3    4    5    6    7    8    9   10   11

PL/SQL procedure successfully completed.
```

```
SQL> CREATE OR REPLACE TRIGGER SYSTEM.EMP_INSERT_TRIGGER
BEFORE INSERT ON SYSTEM.EMPLOYEE
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM Manager_Department
    WHERE oracle_username = USER
      AND dept_id = :NEW.Dept_ID;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'You cannot insert into this department!');
    END IF;
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15   16
Trigger created.
```

```
SQL> BEGIN
    DBMS_RLS.ADD_POLICY(
        object_schema   => 'SYSTEM',
        object_name     => 'SALARY',
        policy_name     => 'SAL_VPD',
        function_schema => 'MANAGER_USER',
        policy_function => 'MANAGERVPDFN',
        statement_types => 'SELECT'
    );
END;
/
  2    3    4    5    6    7    8    9   10   11
PL/SQL procedure successfully completed.

SQL> BEGIN
    DBMS_RLS.ADD_POLICY(
        object_schema   => 'SYSTEM',
        object_name     => 'SALARY',
        policy_name     => 'SAL_VPD_UPDATE',
        function_schema => 'MANAGER_USER',
        policy_function => 'MANAGERVPDFN',
        statement_types => 'UPDATE'
    );
END;
/
  2    3    4    5    6    7    8    9   10   11
PL/SQL procedure successfully completed.

SQL> BEGIN
    DBMS_RLS.ADD_POLICY(
        object_schema   => 'SYSTEM',
        object_name     => 'SALARY',
        policy_name     => 'SAL_VPD_DELETE',
        function_schema => 'MANAGER_USER',
        policy_function => 'MANAGERVPDFN',
        statement_types => 'DELETE'
    );
END;
/
  2    3    4    5    6    7    8    9   10   11
PL/SQL procedure successfully completed.
```

```
SQL> BEGIN
    DBMS_RLS.ADD_POLICY(
        object_schema   => 'SYSTEM',
        object_name     => 'SALARY',
        policy_name     => 'SAL_VPD_DELETE',
        function_schema => 'MANAGER_USER',
        policy_function => 'MANAGERVPDFN',
        statement_types => 'DELETE'
    );
END;
/
  2    3    4    5    6    7    8    9   10   11
PL/SQL procedure successfully completed.
```

```
SQL> CREATE OR REPLACE TRIGGER SYSTEM.SAL_INSERT_TRIGGER
BEFORE INSERT ON SYSTEM.SALARY
FOR EACH ROW
DECLARE
    v_count NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM Employee E
    JOIN Manager_Department M ON E.Dept_ID = M.dept_id
    WHERE E.E_ID = :NEW.E_ID
      AND M.oracle_username = USER;

    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'You cannot insert salary for this employee!');
    END IF;
END;
/
  2    3    4    5    6    7    8    9   10   11   12   13   14   15   16   17   18
Trigger created.
```

4. Encrypt Subsets of the data

Step One:

First, a new table named employee_data was created to act as a subset of sensitive employee information. This table contains fields such as emp_id, emp_name, emp_salary, and emp_nic. At this stage, the sensitive fields (emp_salary and emp_nic) were left as NULL because the values will be encrypted before insertion. Sample employee records were then added for demonstration.

```
SQL> CREATE TABLE employee_data (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(100),
    emp_salary RAW(2000),
    emp_nic RAW(2000)
);  2    3    4    5    6

Table created.

SQL> INSERT INTO employee_data (emp_id, emp_name) VALUES (1, 'Nimal Perera');
INSERT INTO employee_data (emp_id, emp_name) VALUES (2, 'Kavindu Fernando');
1 row created.
```

Step 2: Encrypt and Decrypt Sensitive Fields

In the next step, encryption and decryption functions were implemented using the DBMS_CRYPTO package. The encrypt_data() function was used to securely encrypt sensitive attributes such as salary and NIC before storing them in the table. Later, the decrypt_data() function was applied to retrieve and view the original values in a human-readable format. This ensures that sensitive data remains protected while at rest but can still be accessed securely when needed.

```
SQL> SQL> CREATE OR REPLACE FUNCTION encrypt_data(p_text VARCHAR2)
RETURN RAW IS
BEGIN
    RETURN UTL_RAW.cast_to_raw(p_text);
END;
/  2    3    4    5    6

Function created.

SQL> CREATE OR REPLACE FUNCTION decrypt_data(p_raw RAW)
RETURN VARCHAR2 IS
BEGIN
    RETURN UTL_RAW.cast_to_varchar2(p_raw);
END;
/  2    3    4    5    6

Function created.
```

5. Data Masking.

Created masked views for Employee and Salary tables to protect sensitive information. The Employee view hides phone numbers, partially masks emails, and conceals addresses, while the Salary view hides salary amounts and descriptions. This ensures secure access for users who should not see full private data.

```
SQL> CREATE OR REPLACE VIEW Employee_Masked AS
SELECT
    E_ID,
    First_Name,
    Last_Name,
    'XXXX-XXX-XXXX' AS Phone_Number,
    SUBSTR(Email, 1, 2) || '****' || SUBSTR(Email, INSTR(Email, '@')) AS Email,
    '*****' AS Address,
    Age,
    Gender,
    Dept_ID
FROM Employee;  2    3    4    5    6    7    8    9   10   11   12

View created.

SQL> CREATE OR REPLACE VIEW Salary_Masked AS
SELECT
    Salary_ID,
    E_ID,
    '****' AS Salary_Amount,
    '*****' AS Description
FROM Salary;  2    3    4    5    6    7

View created.
```

6. FGA Policy Implementation.

Configured Fine-Grained Auditing (FGA) on the Employee table to track access to sensitive columns—Phone_Number, Email, and Address—logging every SELECT statement to enhance security and monitor data usage .

```
SQL> BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'EMPLOYEE',
        policy_name     => 'EMPLOYEE_PHONE_AUDIT',
        audit_condition => NULL,
        audit_column    => 'PHONE_NUMBER',
        enable          => TRUE
    );
END;  2    3    4    5    6    7    8    9   10
 11
 12  /

PL/SQL procedure successfully completed.

SQL> BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'EMPLOYEE',
        policy_name     => 'EMPLOYEE_EMAIL_AUDIT',
        audit_condition => NULL,
        audit_column    => 'EMAIL',
        enable          => TRUE
    );
END;
/  2    3    4    5    6    7    8    9   10   11

PL/SQL procedure successfully completed.

SQL> BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'EMPLOYEE',
        policy_name     => 'EMPLOYEE_ADDRESS_AUDIT',
        audit_condition => NULL,
        audit_column    => 'ADDRESS',
        enable          => TRUE
    );
END;
/  2    3    4    5    6    7    8    9   10   11

PL/SQL procedure successfully completed.
```

```
SQL> BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'SALARY',
        policy_name     => 'SALARY_AMOUNT_AUDIT',
        audit_condition => NULL,
        audit_column    => 'SALARY_AMOUNT',
        enable          => TRUE
    );
END;
/  2    3    4    5    6    7    8    9   10   11

PL/SQL procedure successfully completed.

SQL> BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'SALARY',
        policy_name     => 'SALARY_DESCRIPTION_AUDIT',
        audit_condition => NULL,
        audit_column    => 'DESCRIPTION',
        enable          => TRUE
    );
END;
/  2    3    4    5    6    7    8    9   10   11

PL/SQL procedure successfully completed.

SQL> COMMIT;

Commit complete.
```

TASK 3 – Big Data Security

Introduction

Big Data systems are integral to modern enterprises, enabling the processing and analysis of vast amounts of structured, semi-structured, and unstructured data across distributed environments. However, their characteristics scale, velocity, variety, and distributed architecture introduce unique security challenges that differ significantly from traditional, centralized databases. Ensuring robust security in Big Data environments is critical and requires advanced mechanisms to protect confidentiality, integrity, availability, privacy, and enforce fine-grained access control across various data sources, processing frameworks, and storage layers. This section reviews the security landscape of Big Data, compares its requirements with traditional systems, identifies prevalent threats, and discusses mitigation strategies supported by relevant literature.

Security Requirements of Big Data vs. Traditional Databases

The security requirements for Big Data systems differ significantly from those of traditional databases due to architectural and operational distinctions. Below is a comparison of the key differences, supported by various security frameworks and studies (e.g., Marwa, 2022; NIST Big Data Interoperability Framework, 2016).

Security Requirement	Big Data Systems	Traditional Databases
Data Confidentiality	Pervasive encryption at rest (e.g., HDFS Transparent Encryption, object storage encryption) and in transit (e.g., TLS/SSL for node communication). Challenges arise from heterogeneous data formats and distributed clusters.	Encrypting structured data at rest within database files and encrypting data in transit using standard protocols (e.g., TLS). Key management is centralized.
Data Integrity	Maintained through high velocity and frequent movement of data across nodes. Techniques include checksums (e.g., HDFS block checksums), hashing (e.g., SHA-256), and blockchain for immutable logs (Patel et al., 2016).	Enforced using ACID properties, database schemas, constraints, and transaction logs.
Data Availability	Relies on redundancy, replication (e.g., HDFS 3x replication), and fault tolerance (e.g., YARN recovery, Spark RDDs) for availability across large, unreliable clusters.	Achieved via traditional database backups, clustering, and failover mechanisms.
Access Control	Complex, fine-grained, and context-aware access control using tools like Apache Ranger or Sentry to manage permissions across multiple frameworks (Hive, Spark). Role-Based Access Control (RBAC) is insufficient.	Utilizes straightforward RBAC to manage access to tables, views, and stored procedures in centralized systems.
Privacy Protection	Difficult due to scale and variety of data sources (e.g., PII in unstructured formats). Techniques like data anonymization, pseudonymization, and differential privacy are explored (Abdullahi et al., 2020).	Easier to manage in smaller, structured datasets using access controls and data masking for specific columns.

Attack Surface	Larger, with multiple data entry points (APIs, ingestion tools), services (NameNode, DataNodes, ResourceManager), and open-source components with known vulnerabilities (Zhang et al., 2018).	Smaller, confined to database servers and associated network interfaces.

Common Attacks on Big Data Systems and Mitigations

Attack	Description	Mitigations
1. Data Breaches	Unauthorized access to sensitive data due to weak access controls, unpatched vulnerabilities, or compromised credentials.	- **End-to-End Encryption**: Implement AES-256 encryption for data at rest (HDFS Transparent Encryption) and in transit (TLS/SSL) to ensure data remains unreadable if intercepted (Marwa, 2022). - **Multi-Factor Authentication (MFA)** and **Fine-Grained Access Control**: Use MFA for user authentication and tools like Apache Ranger for policy-based access control (Sarkar et al., 2014).
2. Denial of Service (DoS/DDoS)	Attackers overwhelm the Big Data system with excessive traffic, causing performance degradation and resource exhaustion.	- **Resource Management and Isolation**: Use resource managers like Apache YARN with proper queue configurations, quotas, and isolation to prevent malicious or poorly written jobs from overwhelming the cluster. - **Network-Level DDoS Protection**: Deploy network-based protection mechanisms (e.g., traffic filtering, rate limiting) to block malicious traffic before it reaches core services.
3. Data Tampering	Attackers modify or corrupt data during ingestion, storage, or processing, compromising the accuracy of analytics and decision-making.	- **Integrity Verification and Checksums**: Leverage built-in checks like HDFS block checksums and implement custom hashing (e.g., SHA-256) to verify data integrity (Marwa, 2022). - **Immutable Data Logs**: Utilize blockchain-based ledgers or Apache Kafka for immutability in transaction logs (Patel et al., 2016).

4. Man-in-the-Middle (MitM)	Attackers intercept and potentially alter communication between nodes, compromising data confidentiality and integrity.	- **Enforce TLS/SSL Encryption**: Mandate TLS/SSL for inter-node communication and client-service connections within the cluster. - **Endpoint Authentication**: Implement mutual authentication (e.g., Kerberos) to ensure nodes and clients are authenticated before communication.
5. Insider Threats	Authorized users misuse their privileges to steal, manipulate, or leak sensitive data, or disrupt operations.	- **Role-Based Access Control (RBAC) with Least Privilege**: Enforce RBAC and provide users only the permissions necessary for their role. Tools like Apache Ranger/Sentry are useful for granular control (Sarkar et al., 2014). - **Continuous Monitoring and Anomaly Detection**: Use SIEM or UEBA systems to monitor activities and alert on anomalous behaviors, identifying potential insider threats (Abdullahi et al., 2020).

Critical Discussion

Securing Big Data systems is inherently more complex than traditional databases due to the distributed, heterogeneous, and dynamic nature of Big Data environments. Unlike centralized systems, a compromise on one node can potentially propagate across the entire cluster, amplifying the impact of attacks like DoS or data tampering. The sheer volume of data accessible to authorized users also heightens the risk and potential damage from insider threats. Furthermore, the integration of multiple open-source frameworks (e.g., Hadoop, Spark, Kafka, HBase) creates a larger attack surface and requires managing security configurations across various technologies.

Security controls must therefore be |multi-layered|:

Thus, securing Big Data environments requires a multi-layered defense strategy aligned with the CIA triad:

1. **Preventive Controls:** These measures reduce the attack surface, including strong authentication (e.g., Kerberos, MFA), fine-grained access control (Ranger/Sentry), pervasive encryption (at rest and in transit), and network hardening (firewalls, VPNs).
2. **Detective Controls:** These help identify security breaches by monitoring logs, implementing real-time monitoring systems, and using anomaly detection tools (SIEM/UEBA) to spot suspicious activities.
3. **Corrective Controls:** Ensure system recovery in the event of an attack, with incident response procedures, failover mechanisms, and robust backup strategies.

This layered approach is crucial to mitigate the challenges unique to Big Data systems and ensure the confidentiality, integrity, and availability of the data they manage (Marwa, 2022; NIST Big Data Interoperability Framework, 2016).

This layered approach ensures confidentiality, integrity, and availability (CIA triad) in Big Data environments.

References

[1] S. Abdullahi, M. A. Ngadi, and H. I. Hamed, "Security and privacy preservation approaches in big data: A critical survey," *Computer Science Review*, vol. 35, p. 100210, 2020.

[2] E. Marwa, "Security challenges in big data environments," *International Journal of Computer Applications*, vol. 204, no. 24, pp. 1-8, 2022.

[3] National Institute of Standards and Technology (NIST), "NIST Big Data Interoperability Framework: Volume 1, Definitions," NIST Special Publication 1500-1, 2016.

[4] P. Patel, K. Patel, and B. Patel, "A survey of security challenges in big data," *International Journal of Computer Applications*, vol. 139, no. 10, pp. 25-30, 2016.

[5] P. Sarkar, A. Roy, and S. Shome, "A comprehensive survey on security threats and defensive techniques of Hadoop," *International Journal of Computer Applications*, vol. 103, no. 12, pp. 19-28, 2014.

[6] J. Zhang, X. Ding, J. Han, and J. Li, "A survey on security threats and defensive techniques of Hadoop," *Tsinghua Science and Technology*, vol. 23, no. 1, pp. 107-120, 2018.

Appendix

SQL/PLSQL scripts

```
 2. Create Tables
CREATE TABLE Department (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50),
    Head_of_Dept VARCHAR(50),
    Contact_No VARCHAR(15),
    Number_of_Employees INT
);

CREATE TABLE Employee (
    E_ID INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Phone_Number VARCHAR(15),
    Email VARCHAR(100),
    Address VARCHAR(150),
    Age INT,
    Gender VARCHAR(10),
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);

CREATE TABLE Salary (
    Salary_ID INT PRIMARY KEY,
    E_ID INT,
    Salary_Amount DECIMAL(10, 2),
    Description VARCHAR(100),
    FOREIGN KEY (E_ID) REFERENCES Employee(E_ID)
);

CREATE TABLE Project (
    Project_ID INT PRIMARY KEY,
    Dept_ID INT,
    Project_Name VARCHAR(50),
    Duration INT,
    Budget DECIMAL (10, 2),
    FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);

CREATE TABLE Employee_Training (
    Training_ID INT PRIMARY KEY,
    Training_Name VARCHAR(100),
    Cost DECIMAL(10, 2),
```

```sql
    Start_Date DATE,
    End_Date DATE
);

-- Junction table for M:N relationship between Employee and Training
CREATE TABLE Employee_Training_Enrollment (
    E_ID INT,
    Training_ID INT,
    Enrollment_Date DATE DEFAULT SYSDATE,
    Completion_Status VARCHAR(20) DEFAULT 'Enrolled',
    PRIMARY KEY (E_ID, Training_ID),
    FOREIGN KEY (E_ID) REFERENCES Employee(E_ID),
    FOREIGN KEY (Training_ID) REFERENCES Employee_Training(Training_ID)
);

-- 3. Insert Sample Data
-- Department
INSERT INTO Department (Dept_ID, Dept_Name, Head_of_Dept, Contact_No,
Number_of_Employees)
VALUES (1, 'Human Resources', 'Anuradha Perera', '0112456789', 12);
INSERT INTO Department VALUES (2, 'Finance', 'Sanduni Jayasinghe',
'0112789456', 20);
INSERT INTO Department VALUES (3, 'IT Support', 'Kasun Fernando',
'0112987452', 15);
INSERT INTO Department VALUES (4, 'Software Development', 'Dilshan Weerakoon',
'0112236789', 25);
INSERT INTO Department VALUES (5, 'Sales', 'Chathurika Abeykoon',
'0112765432', 30);
INSERT INTO Department VALUES (6, 'Marketing', 'Pradeep Aluthgama',
'0112987645', 18);
INSERT INTO Department VALUES (7, 'Procurement', 'Ruwan Kumara', '0112547896',
10);
INSERT INTO Department VALUES (8, 'Research', 'Sajani Ratnayake',
'0112678912', 8);
INSERT INTO Department VALUES (9, 'Training', 'Malith Silva', '0112435678',
6);
INSERT INTO Department VALUES (10, 'Customer Service', 'Tharushi Senanayake',
'0112893456', 22);

-- Employee
INSERT INTO Employee VALUES (1, 'Nimal', 'Perera', '0771234567',
'nimalp@sliit.lk', 'Colombo', 29, 'Male', 1);
INSERT INTO Employee VALUES (2, 'Kavindu', 'Fernando', '0712345678',
'kavinduf@gmail.com', 'Kandy', 26, 'Male', 2);
INSERT INTO Employee VALUES (3, 'Dilani', 'Jayasinghe', '0753456789',
'dilanij@yahoo.com', 'Galle', 32, 'Female', 3);
INSERT INTO Employee VALUES (4, 'Sajith', 'De Silva', '0764567890',
'sajithd@outlook.com', 'Negombo', 35, 'Male', 4);
```

```sql
INSERT INTO Employee VALUES (5, 'Isuri', 'Gunawardena', '0705678901',
'isurig@gmail.com', 'Matara', 28, 'Female', 5);
INSERT INTO Employee VALUES (6, 'Harsha', 'Kumara', '0776789012',
'harsha.k@abc.lk', 'Kurunegala', 40, 'Male', 6);
INSERT INTO Employee VALUES (7, 'Pavithra', 'Rathnayake', '0747890123',
'pavi.r@ymail.com', 'Anuradhapura', 27, 'Female', 7);
INSERT INTO Employee VALUES (8, 'Chamara', 'Madushanka', '0728901234',
'chamaram@gmail.com', 'Batticaloa', 31, 'Male', 8);
INSERT INTO Employee VALUES (9, 'Nadeesha', 'Wijesinghe', '0789012345',
'nadeesha.w@abc.com', 'Jaffna', 33, 'Female', 9);
INSERT INTO Employee VALUES (10, 'Ruwanthi', 'Bandara', '0710123456',
'ruwanthib@sliit.lk', 'Rathnapura', 30, 'Female', 10);

-- Salary
INSERT INTO Salary VALUES (1, 1, 85000.00, 'HR Executive');
INSERT INTO Salary VALUES (2, 2, 120000.00, 'Accountant');
INSERT INTO Salary VALUES (3, 3, 95000.00, 'IT Officer');
INSERT INTO Salary VALUES (4, 4, 150000.00, 'Software Engineer');
INSERT INTO Salary VALUES (5, 5, 110000.00, 'Sales Executive');
INSERT INTO Salary VALUES (6, 6, 175000.00, 'Marketing Manager');
INSERT INTO Salary VALUES (7, 7, 100000.00, 'Procurement Officer');
INSERT INTO Salary VALUES (8, 8, 140000.00, 'Research Analyst');
INSERT INTO Salary VALUES (9, 9, 90000.00, 'Trainer');
INSERT INTO Salary VALUES (10, 10, 105000.00, 'Customer Care Executive');

-- Project
INSERT INTO Project VALUES (1, 4, 'Payroll System', 6, 500000.00);
INSERT INTO Project VALUES (2, 4, 'Inventory Management', 8, 750000.00);
INSERT INTO Project VALUES (3, 2, 'Budget Planning', 4, 300000.00);
INSERT INTO Project VALUES (4, 6, 'Digital Marketing Campaign', 3, 200000.00);
INSERT INTO Project VALUES (5, 5, 'Sales Tracking System', 5, 400000.00);
INSERT INTO Project VALUES (6, 3, 'Helpdesk Automation', 6, 250000.00);
INSERT INTO Project VALUES (7, 7, 'Supplier Portal', 7, 350000.00);
INSERT INTO Project VALUES (8, 8, 'AI Research', 12, 1000000.00);
INSERT INTO Project VALUES (9, 9, 'Leadership Program', 2, 150000.00);
INSERT INTO Project VALUES (10, 10, 'Customer Feedback App', 4, 450000.00);

-- Employee_Training
INSERT INTO Employee_Training (Training_ID, Training_Name, Cost, Start_Date,
End_Date)
VALUES (1, 'Leadership Training', 50000.00, DATE '2025-01-15', DATE '2025-01-
20');
INSERT INTO Employee_Training (Training_ID, Training_Name, Cost, Start_Date,
End_Date)
VALUES (2, 'Advanced Excel', 20000.00, DATE '2025-02-10', DATE '2025-02-12');
INSERT INTO Employee_Training (Training_ID, Training_Name, Cost, Start_Date,
End_Date)
```

```sql
VALUES (3, 'Cybersecurity Basics', 60000.00, DATE '2025-03-05', DATE '2025-03-
10');
INSERT INTO Employee_Training (Training_ID, Training_Name, Cost, Start_Date,
End_Date)
VALUES (4, 'Public Speaking', 25000.00, DATE '2025-04-01', DATE '2025-04-03');
INSERT INTO Employee_Training (Training_ID, Training_Name, Cost, Start_Date,
End_Date)
VALUES (5, 'Digital Marketing', 45000.00, DATE '2025-05-15', DATE '2025-05-
20');
INSERT INTO Employee_Training VALUES (6, 'Java Programming', 70000.00, DATE
'2025-06-01', DATE '2025-06-15');
INSERT INTO Employee_Training VALUES (7, 'Oracle Database', 90000.00, DATE
'2025-07-01', DATE '2025-07-10');
INSERT INTO Employee_Training VALUES (8, 'Negotiation Skills', 30000.00, DATE
'2025-08-12', DATE '2025-08-14');
INSERT INTO Employee_Training VALUES (9, 'Project Management', 80000.00, DATE
'2025-09-05', DATE '2025-09-12');
INSERT INTO Employee_Training VALUES (10, 'Cloud Computing', 100000.00, DATE
'2025-10-01', DATE '2025-10-07');

-- NEW: M:N Enrollment Data
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (1, 1); --
Nimal -> Leadership
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (1, 3); --
Nimal -> Cybersecurity
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (2, 2); --
Kavindu -> Excel
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (3, 4); --
Dilani -> Public Speaking
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (4, 6); --
Sajith -> Java
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (5, 5); --
Isuri -> Digital Marketing
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (6, 8); --
Harsha -> Negotiation
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (7, 9); --
Pavithra -> Project Mgmt
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (8, 7); --
Chamara -> Oracle DB
INSERT INTO Employee_Training_Enrollment (E_ID, Training_ID) VALUES (9, 10); -
- Nadeesha -> Cloud Computing

COMMIT;
```

```sql
--2. User Roles and Access Control

-- Drop users if they exist (for clean run)
DROP USER system_admin CASCADE;
DROP USER manager_user CASCADE;
DROP USER executive_user CASCADE;

-- 1. System Admin – Full permissions
CREATE USER system_admin IDENTIFIED BY AdminPass123;
GRANT DBA TO system_admin;
GRANT CONNECT, RESOURCE TO system_admin;

-- 2. Manager – Read & Write only
CREATE USER manager_user IDENTIFIED BY ManagerPass123;
GRANT CONNECT, RESOURCE TO manager_user;
-- Grant DML on all tables
GRANT SELECT, INSERT, UPDATE, DELETE ON Department TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Employee TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Project TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Salary TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Employee_Training TO manager_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON Employee_Training_Enrollment TO
manager_user;

-- 3. Executive – Read-only
CREATE USER executive_user IDENTIFIED BY ExecPass123;
GRANT CONNECT TO executive_user;
-- Grant SELECT on all tables
GRANT SELECT ON Department TO executive_user;
GRANT SELECT ON Employee TO executive_user;
GRANT SELECT ON Project TO executive_user;
GRANT SELECT ON Salary TO executive_user;
GRANT SELECT ON Employee_Training TO executive_user;
GRANT SELECT ON Employee_Training_Enrollment TO executive_user;

COMMIT;

3. Manager View

CREATE OR REPLACE VIEW Manager_View AS
SELECT
    E.E_ID,
    E.First_Name,
    E.Last_Name,
    E.Phone_Number,
    E.Email,
    E.Address,
    E.Age,
```

```
    E.Gender,
    D.Dept_Name,
    S.Salary_Amount
FROM Employee E
JOIN Department D ON E.Dept_ID = D.Dept_ID
JOIN Salary S ON E.E_ID = S.E_ID;

-- Test the view (run as manager_user)
-- SELECT * FROM Manager_View;
```

4. VPD Setup

```
-- 1. Create a mapping table for Oracle users to departments
CREATE TABLE Manager_Department (
    oracle_username VARCHAR2(30) PRIMARY KEY,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(Dept_ID)
);

-- Insert the mapping (assuming manager_user manages Dept 1 - HR)
INSERT INTO Manager_Department VALUES ('MANAGER_USER', 1);

-- 2. Create the VPD function using the mapping table
CREATE OR REPLACE FUNCTION MANAGER_USER.MANAGERVPDFN (
    schema_name IN VARCHAR2,
    object_name IN VARCHAR2
) RETURN VARCHAR2
AS
BEGIN
    RETURN 'Dept_ID = (SELECT dept_id FROM Manager_Department WHERE
oracle_username = USER)';
END;

-- 3. Apply VPD policy to Employee table
BEGIN
    DBMS_RLS.ADD_POLICY(
        object_schema    => 'YOUR_SCHEMA_NAME', -- Replace with your actual
schema name (e.g., SYSTEM or your username)
        object_name      => 'EMPLOYEE',
        policy_name      => 'EMP_VPD',
        function_schema  => 'YOUR_SCHEMA_NAME', -- Replace with your actual
schema name
        policy_function  => 'MANAGER_USER.MANAGERVPDFN', -- Specify the full
path
        statement_types  => 'SELECT, INSERT, UPDATE, DELETE' -- Apply to all
DML
    );
END;
```

```oracle
-- 4. Apply VPD policy to Salary table
BEGIN
    DBMS_RLS.ADD_POLICY(
        object_schema    => 'YOUR_SCHEMA_NAME', -- Replace with your actual
schema name
        object_name      => 'SALARY',
        policy_name      => 'SAL_VPD',
        function_schema  => 'YOUR_SCHEMA_NAME', -- Replace with your actual
schema name
        policy_function  => 'MANAGER_USER.MANAGERVPDFN', -- Specify the full
path
        statement_types  => 'SELECT, INSERT, UPDATE, DELETE' -- Apply to all
DML
    );
END;

COMMIT;
```

5. Data Encryption

```oracle
-- 1. Create the table for sensitive data
CREATE TABLE employee_data (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(100),
    emp_salary RAW(2000),   -- encrypted salary
    emp_nic RAW(2000)       -- encrypted NIC/Passport
);

-- 2. Insert sample data (will be encrypted later)
INSERT INTO employee_data (emp_id, emp_name) VALUES (1, 'Nimal Perera');
INSERT INTO employee_data (emp_id, emp_name) VALUES (2, 'Kavindu Fernando');

-- 3. Create an encryption function
CREATE OR REPLACE FUNCTION encrypt_data(p_text VARCHAR2)
RETURN RAW IS
    l_key RAW(32) :=
UTL_RAW.cast_to_raw('ThisIsMy32ByteSecretKeyForAES256!!');
    l_encrypted RAW(32767);
BEGIN
    l_encrypted := DBMS_CRYPTO.encrypt(
                        src => UTL_RAW.cast_to_raw(p_text),
                        typ => DBMS_CRYPTO.ENCRYPT_AES256
                             + DBMS_CRYPTO.CHAIN_CBC
                             + DBMS_CRYPTO.PAD_PKCS5,
                        key => l_key
                    );
    RETURN l_encrypted;
```

```oracle_plsql
END;


-- 4. Create a decryption function
CREATE OR REPLACE FUNCTION decrypt_data(p_raw RAW)
RETURN VARCHAR2 IS
    l_key RAW(32) :=
UTL_RAW.cast_to_raw('ThisIsMy32ByteSecretKeyForAES256!!');
BEGIN
    RETURN UTL_RAW.cast_to_varchar2(
               DBMS_CRYPTO.decrypt(
                   src => p_raw,
                   typ => DBMS_CRYPTO.ENCRYPT_AES256
                        + DBMS_CRYPTO.CHAIN_CBC
                        + DBMS_CRYPTO.PAD_PKCS5,
                   key => l_key
               )
           );
END;

-- 5. Encrypt the data in the table
UPDATE employee_data
SET emp_salary = encrypt_data('85000'),
    emp_nic = encrypt_data('921234567V')
WHERE emp_id = 1;

UPDATE employee_data
SET emp_salary = encrypt_data('120000'),
    emp_nic = encrypt_data('939876543V')
WHERE emp_id = 2;

COMMIT;

-- 6. Demonstrate decryption
SELECT emp_name, decrypt_data(emp_salary) AS decrypted_salary,
decrypt_data(emp_nic) AS decrypted_nic FROM employee_data;
```

6. Data Masking Views

```oracle_plsql
-- Masked Employee view
CREATE OR REPLACE VIEW Employee_Masked AS
SELECT
    E_ID,
    First_Name,
    Last_Name,
    'XXXX-XXX-XXXX' AS Phone_Number,
    SUBSTR(Email, 1, 2) || '****' || SUBSTR(Email, INSTR(Email, '@')) AS
Email, -- Fixed logic
```

```
    '*****' AS Address,
    Age,
    Gender,
    Dept_ID
FROM Employee;

-- Masked Salary view
CREATE OR REPLACE VIEW Salary_Masked AS
SELECT
    Salary_ID,
    E_ID,
    '****' AS Salary_Amount,
    '*****' AS Description
FROM Salary;

-- View masked Employee data
SELECT * FROM Employee_Masked;

-- View masked Salary data
SELECT * FROM Salary_Masked;



--7. FGA Policy Implementation

-- Apply FGA policies to sensitive columns
BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'EMPLOYEE',
        policy_name     => 'EMPLOYEE_PHONE_AUDIT',
        audit_condition => NULL,
        audit_column    => 'PHONE_NUMBER',
        enable          => TRUE
    );
END;

BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'EMPLOYEE',
        policy_name     => 'EMPLOYEE_EMAIL_AUDIT',
        audit_condition => NULL,
        audit_column    => 'EMAIL',
        enable          => TRUE
    );
END;
```

```oracle_plsql
BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'EMPLOYEE',
        policy_name     => 'EMPLOYEE_ADDRESS_AUDIT',
        audit_condition => NULL,
        audit_column    => 'ADDRESS',
        enable          => TRUE
    );
END;

BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'SALARY',
        policy_name     => 'SALARY_AMOUNT_AUDIT',
        audit_condition => NULL,
        audit_column    => 'SALARY_AMOUNT',
        enable          => TRUE
    );
END;


BEGIN
    DBMS_FGA.add_policy(
        object_schema   => 'SYSTEM',
        object_name     => 'SALARY',
        policy_name     => 'SALARY_DESCRIPTION_AUDIT',
        audit_condition => NULL,
        audit_column    => 'DESCRIPTION',
        enable          => TRUE
    );
END;

COMMIT;
```